Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

# Classi generiche
## Progetto di nuove strutture dati

*Corso di Laurea in Ingegneria Informatica*

Anno accademico 2025/2026

**Prof. ENRICO DENTI**

*Dipartimento di Informatica – Scienza e Ingegneria (DISI)*

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA
IL PRESENTE MATERIALE È RISERVATO AL PERSONALE DELL'UNIVERSITÀ DI BOLOGNA E NON PUÒ ESSERE UTILIZZATO AI TERMINI DI LEGGE DA ALTRE PERSONE O PER FINI NON ISTITUZIONALI

# CLASSI GENERICHE

- La notazione di tipo generico **`<T>`** (in Scala: **`[T]`**) si può usare anche per definire classi o interfacce *generiche* rispetto al tipo manipolato.
- Caso tipico: nuovi «contenitori» o *strutture dati*
- ESEMPIO: un nostro *stack*
  - uno stack è una struttura dati ad *accesso LIFO (Last In, First Out)*
  - le due operazioni di aggiunta e rimozione di un elemento dalla (cima della) pila si chiamano usualmente **`push`** e **`pop`**
  - a esse si unisce tipicamente un'operazione per sapere se la struttura sia vuota, spesso chiamata **`isEmpty`**
  - MA.. *di che tipo sono* gli elementi ?
    cosa possiamo / vogliamo poter mettere in questa struttura dati..?
  - Non certo qualunque cosa: meglio *tipizzare* la struttura → **`<T>`**

# UNO STACK GENERICO

- Per definire una struttura dati che manipola un tipo generico **<T>** basta specificarlo *nell'intestazione della classe:*

```
public class MyStack<T> {
   ...
   public void push(T elem){ … }
   public T pop(){ … }
   public boolean isEmpty() { …}
}
```

Java

Di fatto, finora abbiamo definito solo l'interfaccia

  - il tipo **T** è ora noto e usabile *all'interno della classe*
  - NB: lo chiamiamo **MyStack** semplicemente per evitare potenziali name clash con la struttura **java.util.Stack** della JCF
  - Come lo implementiamo?

# STACK GENERICO: POSSIBILI IMPLEMENTAZIONI

- Per implementarlo occorre definire, internamente alla classe, una qualche struttura dati di appoggio
  - opzione #1: una lista di **T**
  - opzione #2: un array di **T**
  - …

Facile, perché le collection sono tipizzate in tutti i linguaggi

In molti linguaggi *meno semplice del previsto*, perché gli array… ☹

- Opzione # 1: implementazione tramite una lista

Java

```
public class MyStack<T> {
  private List<T> storage;
  public MyStack() { storage = new ArrayList<T>(); }
  public void push(T elem){ storage.add(elem); }
  public T pop(){ return storage.remove(storage.size()-1);}
  public boolean isEmpty() { return storage.isEmpty(); }
}
```

NB: in Java, nel costruttore *non* si ripete **<T>**

# STACK GENERICO: POSSIBILI IMPLEMENTAZIONI

- Opzione # 1: implementazione tramite una lista

C#

```csharp
class MyStack<T> {
    private List<T> storage;
    public MyStack() { storage = new List<T>(); }
    public void push(T elem){ storage.Add(elem); }
    public T pop(){ var res = storage[storage.Count-1]; storage.RemoveAt(storage.Count-1); return res; }
    public bool isEmpty() { return storage.Count==0; }
}
```

Scala

```scala
import scala.collection.mutable._

class MyStack[T] {
  private val storage = ListBuffer[T]();
  def push(elem : T) : Unit = { storage += elem; }
  def pop() : T = { return storage.remove(storage.size-1);}
  def isEmpty() : Boolean = { return storage.isEmpty; }
}
```

Kotlin

```kotlin
public class MyStack<T> {
    private val storage = ArrayList<T>();
    public fun push(elem : T) : Unit { storage.add(elem); }
    public fun pop() : T { return storage.removeAt(storage.size-1);}
    public fun isEmpty() : Boolean { return storage.isEmpty(); }
}
```

# STACK GENERICO: MINI-MAIN DI TEST

```java
MyStack<Integer> stack = new MyStack<>();
stack.push(18); stack.push(22); stack.push(34);
while (!stack.isEmpty()) {
    System.out.println(stack.pop());
}
```

Java

```csharp
MyStack<int> stack = new MyStack<int>();
stack.push(18); stack.push(22); stack.push(34);
while (!stack.isEmpty()) {
    Console.WriteLine(stack.pop());
}
```

C#

```scala
val stack = new MyStack[Int]();
stack.push(18); stack.push(22); stack.push(34);
while (!stack.isEmpty()) {
  println(stack.pop());
}
```

Scala

```kotlin
val stack = MyStack<Int>();
stack.push(18); stack.push(22); stack.push(34);
while (!stack.isEmpty()) {
    println(stack.pop());
}
```

Kotlin

**`MyStack`** si usa in modo uniforme alle altre collection.

In particolare, in Java i tipi primitivi sono gestiti tramite le classi wrapper, coi soliti boxing/unboxing automatici.

Politica LIFO:

- inserimento di 18, 22, 34
- estrazione in ordine inverso (34, 22, 18)

```
34
22
18
```

# MIGLIORAMENTO CASCADING & FLUENT INTERFACES

- Con l'interfaccia stabilita, per fare *più push consecutive* occorre scriverle separatamente, una per una:

```
stack.push(18); stack.push(22); stack.push(34);
```

- Sarebbe chiaramente più pratico poterle scrivere di seguito:

```
stack.push(18).push(22).push(34);
```

- Per farlo, basta cambiare leggermente la push:
  - non più tipo di ritorno **`void`**, ma tipo di ritorno.. **`MyStack`** !
  - restituendo poi semplicemente lo stack stesso: **`this`** ☺
- Questo approccio si chiama *cascading* ed è molto comodo: classico "uovo di Colombo", dà luogo a una *fluent interface*

# STACK GENERICO REVISED

- Interfaccia leggermente revisionata

Java

```java
public class MyStack<T> {
   private List<T> storage;
   public MyStack(){ storage = new ArrayList<T>(); }
   public MyStack<T> push(T elem){ storage.add(elem); return this; }
   public T pop(){ return storage.remove(storage.size()-1);}
   public boolean isEmpty() { return storage.isEmpty(); }
}
```

C#

```csharp
public MyStack<T> push(T elem){ storage.Add(elem); return this; }
```

Scala

```scala
def push(elem : T) : MyStack[T] = { storage += elem; return this; }
```

Kotlin

```kotlin
public fun push(elem : T) : MyStack<T> {
    storage[elements++] = elem; return this;
}
```

# STACK GENERICO CON CASCADING: MINI-MAIN DI TEST

Java

```java
MyStack<Integer> stack = new MyStack<>();
stack.push(18).push(22).push(34); // CASCADING
while (!stack.isEmpty()) {
    System.out.println(stack.pop());
}
```

C#

```csharp
MyStack<int> stack = new MyStack<int>();
stack.push(18).push(22).push(34); // CASCADING
while (!stack.isEmpty()) {
    Console.WriteLine(stack.pop());
}
```

Scala

```scala
val stack = new MyStack[Int]();
stack.push(18).push(22).push(34);
while (!stack.isEmpty()) {
  println(stack.pop());
}
```

Kotlin

```kotlin
val stack = MyStack<Int>();
stack.push(18).push(22).push(34);
while (!stack.isEmpty()) {
    println(stack.pop());
}
```

**`MyStack`** si usa in modo uniforme alle altre collection.

In particolare, in Java i tipi primitivi sono gestiti tramite le classi wrapper, coi soliti boxing/unboxing automatici.

Politica LIFO:

- inserimento di 18, 22, 34
- estrazione in ordine inverso (34, 22, 18)

```
34
22
18
```

# STACK GENERICO: POSSIBILI IMPLEMENTAZIONI

- Opzione # 2: implementazione tramite un array
  - ci si potrebbe aspettare una cosa del tutto analoga
  - MA emerge un problema: in molti linguaggi, un array di **T** si può dichiarare, ma *non istanziare!* ☹

Java

```
public class MyStackBis<T> {
   private T[] storage;          // La dichiarazione è OK…
   private int elements;
   public MyStackBis(int size){
       storage = new T[size];   // ..MA l'allocazione generica NO!
       elements = 0;
   }
   public void push(T elem){ storage[elements++] = elem; }
   public T pop(){ return storage[--elements]; }
   public boolean isEmpty() { return elements==0; }
}
```

Non compila!! ☹☹

# IMPLEMENTAZIONE CON ARRAY IN LINGUAGGI JVM-BASED

- Perché questo?

  Il problema deriva dalla tecnica di *type erasure* adottata per implementare i tipi generici senza cambiare la JVM
  - in Java e negli altri linguaggi su JVM, i tipi generici sono una *illusione ottica* creata dal compilatore
  - nel codice compilato spariscono, perché *il bytecode non ha quel concetto* e *non ne mantiene alcuna rappresentazione a runtime*
  - sotto, tutto diventa `Object` (o un altro tipo «adatto»)!
- Allocare array di tipi «sconosciuti» è quindi impedito perché sarebbe *impossibile garantire la type safety* a runtime
  - si può bypassare il problema con opportuni workaround, diversi da linguaggio e linguaggio
- Quando possibile, molto meglio usare liste o altre collection!

# IMPLEMENTAZIONE CON ARRAY IN C#

- Nella macchina virtuale CLR di .NET e C#, invece, *i tipi generici esistono anche a run-time*
  - nessuna type erasure ☺ ☺
  - il bytecode MSIL conosce e reifica il concetto di tipo generico e ne mantiene una corretta rappresentazione a runtime

C#

```
class MyStackBis<T> {
    private T[] storage;
    private int elements;

    public MyStackBis(int size){
        storage = new T[size]; // allocazione generica
        elements = 0;
    }

    public void push(T elem){ storage[elements++] = elem; }
    public T pop(){ return storage[--elements]; }
    public bool isEmpty() { return elements==0; }
}
```

Compila senza problemi ☺☺

# IMPLEMENTAZIONE CON ARRAY IN LINGUAGGI JVM-BASED

- Cosa fa la tecnica di type erasure per la JVM?
  - cancella il tipo **`T`**, sostituendolo nel bytecode con **`Object`**
    → l'informazione sul tipo effettivo *si perde* a runtime
  - ergo, il compilatore non accetta l'allocazione generica **`new T[…]`** perché non può generare codice che garantisca il check a runtime
- Come si può bypassare il problema?
  1. accettando che non ci sia alcuna verifica ☺ ☹ e allocando noi, intenzionalmente, un array di **`Object`** (o altro tipo «adatto») ed effettuando contestualmente i «giusti» cast (cough, cough..)
     → MA è un «toppa» pasticciata, che funziona solo parzialmente ☹
  2. oppure, più elegantemente, predisponendo una factory che sfrutti internamente il metodo **`Arrays.copyOf`**, a cui passare un array «farlocco» del giusto tipo (come nel metodo **`toArray`** di **`List`**)

# APPROCCIO #1: ESEMPIO

Java

```
class PoorGenericArray<T> {
   private T[] array;
   public PoorGenericArray(int size){
        array = (T[]) new Object[size];
   }
   public T get(int index){ return array[index]; }

   public void set(int index, T elem){ array[index] = elem; }

   public int size(){ return array.length; }
}
```

- nominalmente, questa classe è corretta, compila e funziona
- tuttavia, internamente **`array`** rimane un array di **`Object`** e questo può creare disastri *se l'array viene reso visibile all'esterno della classe*

# APPROCCIO #1: ESEMPIO

Un possibile main (corretto e senza problemi):

Java

```
var arrayOfStrings  = new PoorGenericArray<String>(5);
var arrayOfIntegers = new PoorGenericArray<Integer>(9);

arrayOfStrings.set(0,"pluto"); arrayOfStrings.set(1, "paperino");
arrayOfStrings.set(3, "paperone");

for (int i=0; i<arrayOfStrings.size(); i++)
   System.out.print(arrayOfStrings.get(i) + " ,");

arrayOfIntegers.set(0, 22); arrayOfIntegers.set(1, 33);
arrayOfIntegers.set(2, 44);

for (int i=0; i<arrayOfStrings.size(); i++)
   System.out.print(arrayOfIntegers.get(i) + " ,");
```

```
pluto, paperino, null, paperone, null,
22, 33, 44, null, null,
```

# APPROCCIO #1: LO STACK REVISED

Java

```java
class MyStackBis<T> {
    private T[] storage;        // La dichiarazione è OK…
    private int elements;
    public MyStackBis(int size){
    // storage = new T[size];   // ..MA l'allocazione generica NO!

    storage = (T[]) new Object[size];   // workaround
    elements = 0;
    }
}
```

Java

- si alloca per forza di cose un array di `Object`…
- ..ma subito dopo lo si converte con un *cast* nel giusto tipo `T[]`

Scala

```scala
class MyStackBis[T](val capacity: Int) {
  private var elements : Int = 0;

  private val storage : Array[T] = new Array[Any](capacity).asInstanceOf[Array[T]];

  def push(elem : T) : MyStackBis[T] = {
    storage(elements) = elem; elements += 1; return this;
  }
  def pop() : T = {
  def isEmpty() : B
}
```

Scala: idem

Kotlin: idem (serve il *nullable type* perché l'array viene creato con le celle tutte null)

```kotlin
public class MyStackBis<T>(capacity: Int) {
    private var elements : Int = 0;

    private val storage : Array<T?> = arrayOfNulls<Any?>(capacity) as Array<T?>
    // workaround per allocazione generica!

    public fun push(elem : T) : MyStack<T> {
        storage[elements++] = elem; return this;
    }
    public fun pop() : T { return storage[--elements] as T; }
    public fun isEmpty() : Boolean { return elements==0; }
}
```

Kotlin

# APPROCCIO #1: LO STACK REVISED

Java

```java
MyStackBis<Integer> stackBis = new MyStackBis<>(5);
stack.push(18).push(22).push(34); // CASCADING
while (!stackBis.isEmpty()) {
    System.out.println(stackBis.pop());
}
```

C#

```csharp
MyStackBis<int> stackBis = new MyStackBis<int>(5);
stack.push(18).push(22).push(34); // CASCADING
while (!stackBis.isEmpty()) {
    Console.WriteLine(stackBis.pop());
}
```

Scala

```scala
val stackBis = new MyStackBis[Int](5);
stackBis.push(18); stackBis.push(22); stackBis.push(34);
while (!stackBis.isEmpty()) {
  println(stackBis.pop());
}
```

Kotlin

```kotlin
MyStackBis<Integer> stackBis = new MyStackBis<>(5);
stack.push(18).push(22).push(34); // CASCADING
while (!stackBis.isEmpty()) {
    System.out.println(stackBis.pop());
}
```

# APPROCCIO #1: PROBLEMA

- Se però l'array viene *reso visibile all'esterno* della classe, disastro!
- Formalmente il tipo è corretto (grazie al cast che «maschera» le cose) ma *in realtà esce un array di* **`Object`** e quindi a runtime… BOOM!

Java

```
class PoorGenericArray<T> {
   ...
   public T[] getArray(){ // pericolosissimo!
        return array;
   }
}
```

Java

```
String[]  v1 = arrayOfStrings.getArray();   // BOOM!
Integer[] v2 = arrayOfIntegers.getArray();  // BOOM!
```

```
Exception in thread "main" java.lang.ClassCastException: class
[Ljava.lang.Object; cannot be cast to class [Ljava.lang.String;
```

# APPROCCIO #2: LA FACTORY

- Un approccio *nettamente migliore* è predisporre una factory che sfrutti internamente il metodo **`Arrays.copyOf`**, a cui passare un array «farlocco» del giusto tipo (come nel metodo **`toArray`** di **`List`**)

Java

```
public class GenericArrayFactory {
    @SafeVarargs
    public static <T> T[] of(int size, T... dummyArray){
        return Arrays.copyOf(dummyArray, size);
    }
}
```

- in questo modo, **`copyOf`** può usare l'array **`dummy`** (formalmente, un varargs) come template per creare il «vero» array del «giusto» tipo ☺
- l'array che si ottiene *non è più di Object, ma effettivamente di T* e può quindi essere usato in modo pulito in ogni contesto ☺☺
- <u>ulteriore benefit</u>: essendo anche formalmente un array, si possono usare le parentesi quadre [ ] e tutti i metodi degli array! ☺☺

# APPROCCIO #2: ESEMPIO

Un possibile main:

```
var arrayOfStrings = GenericArrayFactory.of(5, new String[0]);
var arrayOfIntegers = GenericArrayFactory.of(9, new Integer[0]);
arrayOfStrings[0] = "pluto"; arrayOfStrings[1] = "paperino";
arrayOfStrings[3] = "paperone";
System.out.println(String.join(",", arrayOfStrings));
arrayOfIntegers[0] = 22; arrayOfIntegers[1] = 33;
arrayOfIntegers[2] = 44;
System.out.println(Arrays.toString(arrayOfIntegers));
```

Java

Array dummy, template per il tipo effettivo da creare

```
pluto,paperino,null,paperone,null
[22, 33, 44, null, null, null, null, null, null]
```